- Study Case 1 **(Kita adalah Sepasang Sepatu)**

Lamine adalah seorang pemain bola untuk tim Persija Jakarta. 3 hari lagi ada pertandingan bola yang sangat penting. Persija Jakarta akan bertanding melawan Manchester United. Lamine baru sadar ternyata ada sebuah masalah, sepatu bola Lamine rusak minggu lalu karena la **suka menendang tiang gawang ataupun teman satu timnya**. Lamine belum punya sepatu saat ini, karena Persija tidak mau lagi untuk memberinya sepatu dan uang pribadinya telah habis untuk **membeli bitcoin**. Orang tua lamine memiliki inisiatif untuk membangun toko sepatu agar lamine tidak akan pernah kehabisan sepatu karena ia hancurkan sendiri. Namun, kedua orang tua lamine tidak bisa membuat **sistem manajemen toko sepatu**. Oleh karena itu, orang tua lamine meminta bantuan **KAMU** untuk membuatkannya sebuah program dengan bahasa **C++** dengan konsep **Object Oriented Programming**. Orang tua lamine berharap besar kepada kamu untuk anaknya tercinta.

Ketentuan Umum

1. **Wajib Gunakan konsep OOP dengan syarat minimal ada dari yang disebutkan dibawah ini**:

- Class & Object
- Attribute/Property & Method/Behavior
- Constructor
- Setter & Getter
- Encapsulation
- Inheritance
- Access Modifier/ Visibility

2. **Mengimplementasikan CRUD (Create, Read, Update, Delete)**

Notes = **Wajib membuat 1 fungsionalitas** dari **setiap implementasi** dibawah ini, jika membuat lebih dari 1 akan menjadi nilai tambah. Misalkan, implementasi ***Create***, kalian boleh memilih untuk menambah jumlah karyawan/ stok produk di gudang maupun hal lainnya. Karena, yang dilihat adalah implementasinya. Dibawah ini

merupakan contoh yang dapat diambil, diperbolehkan untuk menggunakan/ menambah fungsionalitas lain.

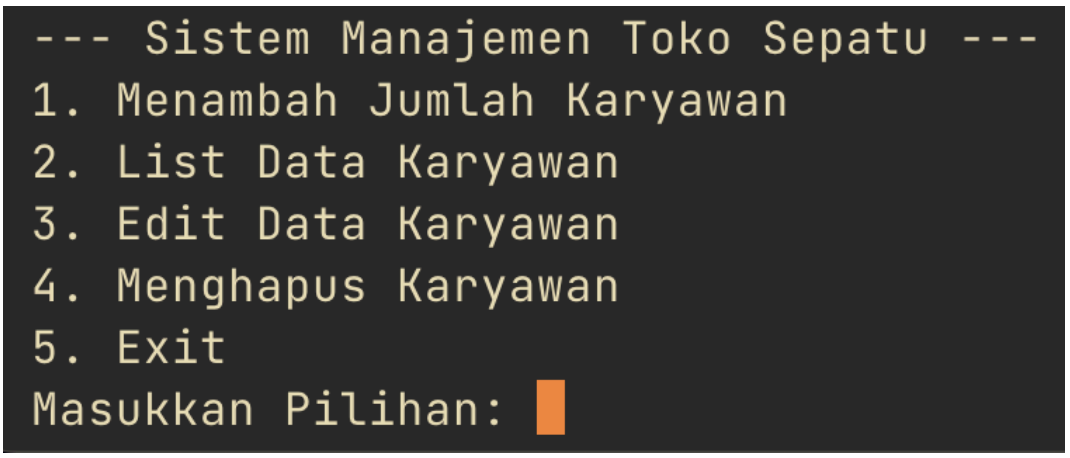
- Create : Menambah (jumlah karyawan/ stok produk di gudang)
- Read : List (produk yang saat ini di display/ data karyawan)
- Update : Memperbarui/ edit (produk apa dari gudang yang ingin di display/ karyawan yang sudah tidak bekerja)
- Delete : Menghapus (barang dari gudang/ karyawan yang sudah tidak bekerja)

3. **Dilarang menggunakan Standard Template Library (STL)**

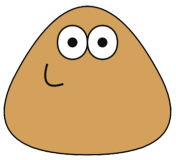
Class Details & Attribute

1. Karyawan (Class) = Kasir, Sales
2. produkDisplay(Class) = Sepatu, Kaos Kaki
3. pekerjaGudang (Class) = ManajerGudang, Supplier

Output yang diharapkan: (berupa saran, tidak sama juga gapapa)

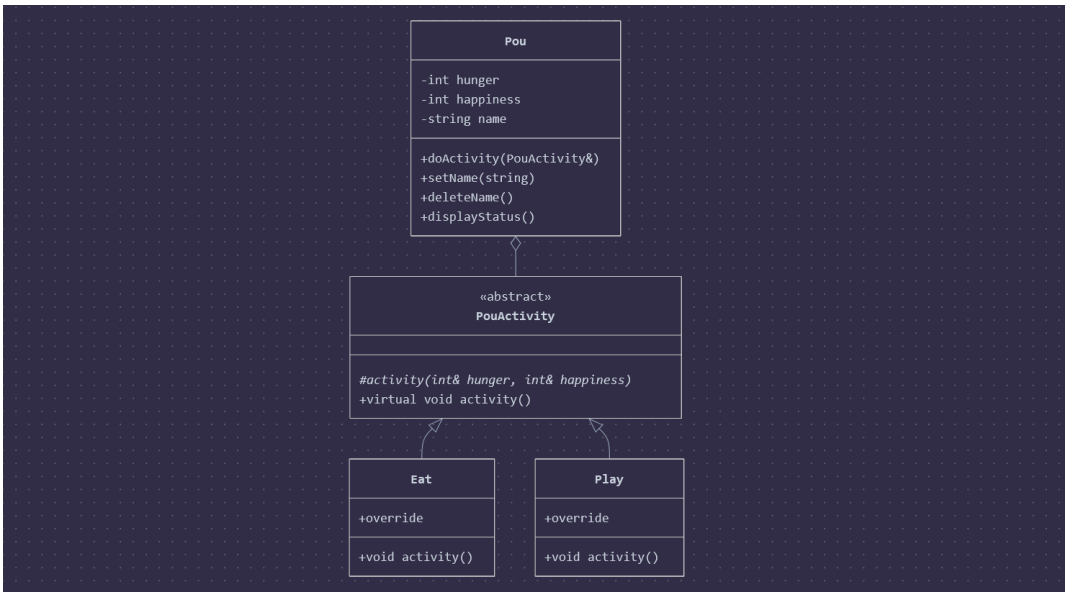


- Study Case 2 **(Pou Simulator)**



Pou is your cute alien pet who LOVES to eat and play!

Requirements



Main class (Pou)	1. Buat 3 attribute dengan visibility private: ○ hunger (integer): Tracks hunger level ○ happiness (integer): Tracks happiness level ○ name (string): Menyimpan data nama pou 2. Buat default constructor Pou dengan attribute: ○ hunger = 50 ○ happiness = 5 ○ name = “No name” 3. Buat method: ○ Setter setName untuk set data pada attribute name sesuai input user (berupa string) ○ Setter deleteName untuk set data pada attribute name kembali menjadi “No name” ○ function untuk menjalankan eat/play pada derived class yang *inherited* dari base class ○ function display status i. Status hunger ii. Status happiness

	iii. Name
Base Class (doActivity)	1. Virtual method: ○ **Functionality**: Mencetak “Pou is doing something...”
Derived Class 1 (Eat)	1. Inherit dari Base Class 2. Override virtual method pada base class: ○ Memanggil Base Class ○ **Functionality**: Menambahkan happiness (+10), mengurangi hunger (-10) ○ **Functionality**: Mencetak “Pou is eating!”
Derived Class 2 (Play)	1. Inherit dari Base Class 2. Override virtual method pada base class: ○ Memanggil Base Class ○ **Functionality**: Menambahkan happiness (+10), menambahkan hunger (+15) ○ **Functionality**: Mencetak “Pou is playing!”
main()	1. Buat object myPou dari main class (Pou) 2. Buat object eating dari derived class 1 (Eat) 3. Buat object playing dari derived class 2 (Play) 4. Buat variabel choice 5. Buat loop menu pilihan dengan mengambil variable choice dari user: ○ 1. Eat ■ Contoh: myPou.doActivity(eating) ○ 2. Play ■ Contoh: myPou.doActivity(playing) ○ 3. Give Pou a name ■ Ambil input dari user ■ Contoh: myPou.setName(pouName) ○ 4. Delete Pou’s name ■ Contoh: myPou.deleteName() ○ 5. Check Pou’s status ■ Contoh: myPou.displayStatus() ○ 6. Exit

Notes:

- Tidak perlu memory allocation, create dan delete name di soal ini memang bukan literally create/delete tapi update menggunakan setter
- Selain yang disebutkan di atas, bebas untuk menambahkan dekorasi/functionality lainnya tetapi tidak masuk penilaian.

- Time consideration: Tidak wajib, tetapi sangat boleh untuk menggunakan konsep OOP selain yang telah disebutkan di requirement
    - Contohnya, tidak wajib membuat setter dan getter untuk attribute happiness dan hunger, tapi kalau ada waktu lebih boleh ditambah.

- Study Case 3 **(Aroma Harapan di tengah Tandusnya Windward Plains)**

Setelah bertahun-tahun mengabdikan diri sebagai pemburu monster atas permintaan guild, **Nata** akhirnya meraih gelar sebagai hunter terbaik. Namun, di balik prestasi gemilang itu, Nata menyimpan impian yang jauh lebih sederhana namun bermakna; **membuka restoran yang dapat menjadi sumber inspirasi sekaligus tempat bernaung bagi siapa saja yang membutuhkan**. Impian ini berakar dari masa kecilnya, saat Nata pernah ditolong oleh seorang hunter asing yang kini menjadi panutannya. Sosok tersebut kerap memasak hidangan yang tidak hanya lezat, tetapi juga mampu menghangatkan hati.

Sebagai sahabat Nata, kamu merasa iba melihat perjuangannya dan bertekad untuk membantu mewujudkan impian tersebut. Berbekal pengetahuan baru tentang **Object Oriented Programming**, kamu pun memutuskan untuk membuat sebuah sistem yang dapat **mengelola data menu** di restoran Nata. Kini, saatnya kamu berperan dalam membantu Nata mencapai impian terbesarnya.

Ketentuan Umum

1. **Wajib Gunakan konsep OOP dengan syarat minimal ada dari yang disebutkan dibawah ini**:
    - Class & Object
    - Attribute/Property & Method/Behavior
    - Constructor
    - Setter & Getter
    - Encapsulation
    - Inheritance
    - Polymorphism

2. **Mengimplementasikan CRUD (Create, Read, Update, Delete)**

    Notes = **Wajib membuat 1 fungsionalitas** dari **setiap implementasi** dibawah ini, jika membuat lebih dari 1 akan menjadi nilai tambah. Misalkan, implementasi ***Create***, kalian

boleh memilih untuk menambah menu baru maupun hal lainnya. Karena, yang dilihat adalah implementasinya. Dibawah ini merupakan contoh yang dapat diambil, diperbolehkan untuk menggunakan/ menambah fungsionalitas lain.

- Create : Menambah menu baru
- Read : List (daftar menu yang tersedia)
- Update : Memperbarui/ edit (nama menu / harga menu/)
- Delete : Menghapus food yang tidak lagi tersedia

3. **Dilarang menggunakan Standard Template Library (STL)**

Class Details & Attribute

1. Menu (Class) = name, price
2. Food (Class) = portionSize(kecil/sedang/besar), buffEffect(low, medium, high)
3. Drink (Class) = volume(kecil/sedang/besar), sweetness(less, normal,extra)

- Study Case 4 **(Ingin lulus strukdat)**
  - Requirements

Class hero	- Protected - string name; - int hp; - int str; - int intel; - int cha; - int gold; - bool hasPotion; - Constructor : - Semua stat = 0 - has potion = false - Function untuk register - input nama - input stat(jumlah stat semua maksimal 10) - Virtual method untuk print stats
Class HeroActivity (Derived from Hero)	- function printstats (override dari class hero) - function latihanotot -> str +2 - function belajar -> int +2 - function tidoo -> hp +2 - function mewing -> cha +2 - function fightenemies -> hp -3, gold +10 - function buypotion -> haspotion = true, gold -20, print “potion lulus modul 1 strukdat berhasil dibeli” - function usepotion -> print “Congrats”, exit the program
main()	- hero.registerhero() - while loop minta variabel dari user - 1. Latihan otot - 2. Belajar - 3. Tidoo; - 4. Mewing - 5. Fight enemies - 6. Beli Potion 'lulus modul 1 strukdat' - 7. Use potion - 8. Tampilkan status hero - 9. Keluar

Note :

- Selain diatas boleh ditambahkan(tidak masuk penilaian)

- Yang wajib ada :
    - Class & Object
    - Attribute/Property & Method/Behavior
    - Constructor
    - Encapsulation
    - Inheritance
    - Overriding